Multi-Tool Research Agent

A Grounded, Tool-Calling LLM Agent with Memory,
Caching, Cost Control, and Evaluation

Technical Architecture & Implementation Report

Project 2 — AI Engineering Roadmap (Agentic AI Foundations)

Stack: Python 3.12 · OpenAI (gpt-4o-mini) · Pydantic · FastAPI · SQLite · Docker

Tools: Web Search · arXiv · Wikipedia · YouTube Transcripts · Calculator

June 2026

Table of Contents

1. Executive Summary

- What the system is
- Headline results

2. Problem Statement & Motivation

- Why grounding matters
- Why tool-calling is the foundation of agentic AI

3. Core Concepts

- Tool calling
- Native tool calling vs. ReAct
- JSON Schema as the tool contract
- The ToolResult envelope

4. System Architecture

- The request lifecycle
- Module map & dependency graph
- Control flow of the agentic loop

5. Component Deep-Dives

- Configuration layer
- Schemas & data contracts
- The five tools
- Tool registry & dispatch
- Conversation memory
- The agent loop & cost control
- Caching layer
- HTTP API

6. Production Engineering

- Graceful degradation
- Hard timeouts & the threading model
- Cost caps
- Caching strategy & staleness
- Security posture

7. Evaluation

- Methodology & design decisions
- Results
- What the eval surfaced

8. Deployment

- Containerization
- Continuous integration
- Hosting

9. Limitations & Future Work

10. Engineering Decisions & Problems Encountered

- Tool-calling reliability & prompt design
- System design & data contracts
- Concurrency & reliability
- Memory & cost control
- Caching
- Security
- Evaluation integrity
- Engineering process

11. Conceptual Review: Design Q&A

- Tool calling & ReAct
- Conversation memory
- Cost control
- Caching
- Evaluation design

12. Appendix: Glossary & File Index

1. Executive Summary

1.1 What the system is

The Multi-Tool Research Agent is a Large Language Model (LLM) application that answers questions by deciding, per query, which external tool to invoke, executing that tool, reading the result, and synthesizing a final answer grounded in real retrieved data with inline source citations. It is not a single canned API call: the model drives a multi-step loop, choosing one of five tools — general web search, arXiv academic search, Wikipedia lookup, YouTube transcript retrieval, and a sandboxed calculator — or answering directly from its own knowledge when no tool is warranted.

The system is exposed through three interchangeable interfaces backed by a single core: a one-shot command-line invocation, an interactive REPL with persistent conversation memory, and an HTTP API suitable for deployment. The same agent logic serves all three.

1.2 Headline results

Dimension	Result
Tool-selection accuracy	96.0% (24/25) on a labeled evaluation set
Tools integrated	5 (web, arXiv, Wikipedia, YouTube, calculator)
Cost per request	Typically under US$0.002; hard cap at US$0.05
Failure handling	Structured errors + automatic fallback to alternate tools
Caching	SQLite, 24-hour TTL, keyed on validated arguments
Memory	Token-budgeted sliding window preserving tool-call integrity
Interfaces	CLI, interactive REPL, HTTP API (FastAPI), Docker container

2. Problem Statement & Motivation

2.1 Why grounding matters

An LLM is a frozen text predictor. Its knowledge ends at a training cutoff, and it has no intrinsic access to the live world: it cannot report today's news, cannot retrieve a paper published last week, and cannot compute an exact arithmetic result reliably. Asked a time-sensitive or computational question, an ungrounded model produces fluent but potentially stale or fabricated output. The defining failure mode observed early in development was precisely this — the model confidently answering a question about current regulation with information frozen at its training cutoff, presented as if current.

Grounding addresses this by routing the model's information needs through external tools whose outputs are current and verifiable, then requiring the model to cite the specific source of each claim. The result is a system that "goes and finds out" rather than "guesses from memory."

2.2 Why tool-calling is the foundation of agentic AI

The single most important capability in this project is tool calling: the mechanism by which an LLM emits a structured request to invoke a function, the application executes it, and the result is fed back into the conversation. The model never executes anything itself; it only *decides* what to do based on the tool descriptions it is given. This decision — choose a tool, observe the result, decide the next step — is the atomic unit of agentic behavior. Every more sophisticated agent (multi-agent systems, planners, reasoning loops) is built on this same primitive. Mastering it is the prerequisite for everything that follows in agentic AI.

3. Core Concepts

3.1 Tool calling

Modern LLMs are post-trained to recognize a list of available tools — each described by a name, a natural-language description, and a JSON Schema for its arguments — and to emit a structured JSON request when a tool is appropriate. The application detects this request, runs the corresponding function, and appends the result to the message history as a dedicated message. The model then continues, now able to read the tool's output.

Crucially, tool selection is statistical, not deductive. The model pattern-matches the user's intent against the tool descriptions and argument names. Vague descriptions produce poor routing; precise, example-laden descriptions produce reliable routing. The tool descriptions and the system prompt are therefore the two primary levers controlling agent behavior.

3.2 Native tool calling vs. ReAct

An earlier paradigm, ReAct, prompted the model to emit reasoning and actions as free text in a strict textual format ("Thought / Action / Observation"), which the application parsed with regular expressions. This is brittle — a single malformed line breaks the parser — and verbose. Native tool calling, used throughout this project, instead relies on the model emitting structured JSON validated by the provider's SDK. It is more reliable, supports parallel tool calls, and eliminates fragile text parsing. ReAct remains relevant only for models without native tool-calling support.

3.3 JSON Schema as the tool contract

Each tool's arguments are defined as a JSON Schema — the formal contract between the model and the application. In this project, those schemas are authored as Pydantic models and converted to the provider's tool-definition format programmatically. This yields a single source of truth: the same Pydantic class both (a) generates the schema the model sees and (b) validates the arguments the model returns before the tool runs. Tight schemas — with enumerated value sets, bounded numeric ranges, and explicit descriptions — reduce hallucinated or malformed arguments.

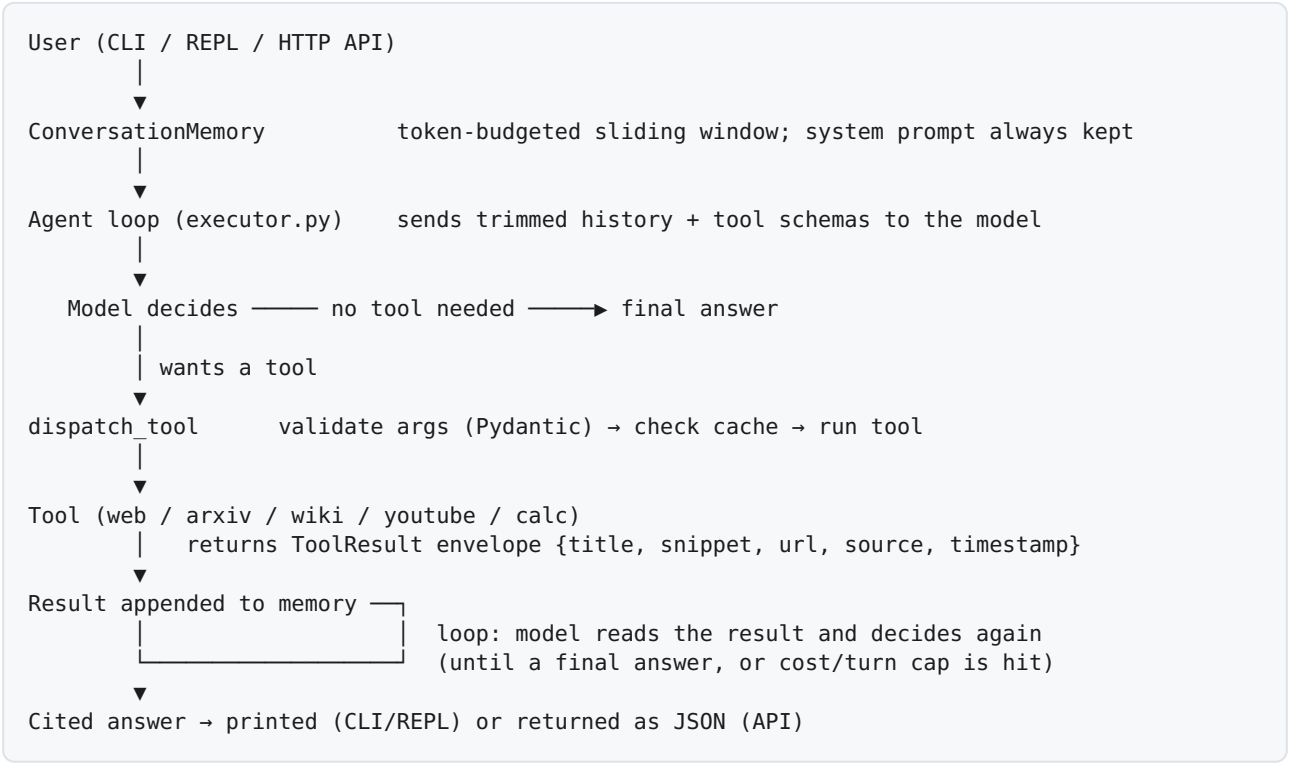
3.4 The ToolResult envelope

Every retrieval tool returns its results in one uniform structure, the `ToolResult` envelope, carrying five fields: `title`, `snippet`, `url`, `source`, and `timestamp`. Normalizing heterogeneous tool outputs (web hits, papers, articles, transcripts) into one shape yields three benefits: citations become trivial (the model is instructed to cite the `url` of each claim), freshness is explicit (the `timestamp` tells the model how current the data is), and the underlying provider becomes swappable without changing the model-facing contract.

4. System Architecture

4.1 The request lifecycle

A single question traverses the system as follows. The diagram traces one query from entry to a cited answer, including the loop that distinguishes an agent from a single tool call.



The defining characteristic is the feedback loop: a tool result is never the end of processing. It is appended to the conversation and the model is invoked again, now able to decide whether the answer is complete, whether another tool is needed, or whether to retry. The loop is bounded by a hard turn cap (`MAX_TURNS = 5`) and a per-request cost cap, guaranteeing termination.

4.2 Module map & dependency graph

The codebase is organized as an installable package with a strict, acyclic dependency graph. Lower layers know nothing of higher ones.

```
scripts/agent_cli.py        entry point: CLI + REPL
        │
        ▼
app/agent/executor.py       the agentic loop
   ├── app/agent/prompts.py  system prompt construction
   ├── app/memory.py         ConversationMemory (token-budgeted window)
   └── app/tools/            registry + 5 tool implementations
            │
            ▼
        app/schemas.py       Pydantic arg models + ToolResult + error_payload
        app/config.py        constants, executor, OpenAI client  (bottom of graph)

app/api/main.py             FastAPI wrapper → calls executor.run_agent
eval/tool_accuracy.py       evaluation harness → calls executor.run_agent
```

File	Responsibility
`app/config.py`	Shared constants, timeouts, the thread-pool executor, the OpenAI client, environment loading. No internal dependencies.
`app/schemas.py`	All Pydantic argument models, the ToolResult envelope, and the uniform error payload helper.
`app/memory.py`	Token counting and the ConversationMemory sliding-window class.
`app/tools/*.py`	One file per tool (web, arxiv, wiki, youtube, calc) plus the registry.
`app/agent/prompts.py`	System prompt: date anchor, tool-selection rules, response-shape contract.
`app/agent/executor.py`	The agentic loop, cost tracking, tool dispatch orchestration.
`app/cache.py`	SQLite-backed tool-result cache with TTL.
`app/api/main.py`	FastAPI application exposing `/health` and `/query`.
`scripts/agent_cli.py`	Thin CLI/REPL entry point.
`eval/tool_accuracy.py`	Tool-selection accuracy evaluation harness.

4.3 Control flow of the agentic loop

For a representative multi-tool query — "Compare Marie Curie's work (Wikipedia) with recent arXiv papers on radioactivity" — the loop executes as follows:

1. The question becomes a `user` message and enters memory.
2. The trimmed message history plus the tool schemas are sent to the model.
3. The model emits a tool call for `search_wikipedia`; the application validates the arguments, checks the cache, runs the tool, and appends the `ToolResult` JSON as a `tool` message.
4. The loop repeats; the model now emits a tool call for `search_arxiv`; the result is appended.

5. The loop repeats; with both results in context, the model produces a final answer with no further tool calls, citing both sources. The loop terminates and the answer is returned.

5. Component Deep-Dives

5.1 Configuration layer

All cross-cutting constants live in one dependency-free module: the model identifier, the loop cap, per-tool timeout values, token cost rates, the cost cap, the shared thread pool, and the single reused OpenAI client. Centralizing these eliminates magic numbers scattered across the codebase and provides one obvious place to tune behavior. Environment variables (notably the API key) are loaded before the client is constructed.

5.2 Schemas & data contracts

Five Pydantic models define the tool inputs, each with bounded and enumerated fields where appropriate — for example, the arXiv category is constrained to a fixed set of valid subject codes, and result counts are bounded to sane ranges. The `ToolResult` envelope defines the uniform output. A shared `error_payload` helper produces the single error JSON shape every tool returns on failure, ensuring the model encounters a consistent, recognizable failure signal.

5.3 The five tools

Tool	Backend	Notes
`search_web`	DuckDuckGo (ddgs)	General current-events search; recency filter mapped from human labels to provider codes.
`search_arxiv`	arXiv API	Academic preprints; category filtering and client-side date filtering; abstracts truncated to control context size.
`search_wikipedia`	wikipedia-api	Encyclopedic facts; descriptive User-Agent required by Wikipedia; long articles capped.
`fetch_youtube_transcript`	youtube-transcript-api	Accepts URL or bare ID; transcript capped to control context size.
`calculator`	numexpr	Sandboxed math evaluation — never uses `eval()`, eliminating the code-execution attack surface.

Each network tool follows an identical resilience pattern: the work is submitted to a shared thread pool and awaited with a hard timeout; on success it returns a JSON array of `ToolResult` objects; on any failure it returns the uniform error payload.

5.4 Tool registry & dispatch

The registry converts each Pydantic model into the provider's tool-definition format, assembles the list of tools sent to the model, and exposes a single `dispatch_tool` entry point. Dispatch performs three steps in order: validate the model's raw arguments against the Pydantic schema; consult the cache; on a miss, run the tool and conditionally store the result. The conditional store is a deliberate safeguard —

error results are never cached, preventing a transient failure from being replayed for the entire cache lifetime.

5.5 Conversation memory

The `ConversationMemory` class maintains the full message history and, on each request, returns a trimmed copy that fits within a configurable token budget. Three invariants are enforced: the system message is never dropped; messages are kept newest-first (recent context is most relevant); and an assistant tool-call message and its corresponding tool-result messages are always trimmed as a single group. The last invariant is mandatory — the provider's API rejects any tool-result message that lacks its preceding tool-call, so partial groups are never produced. When the budget is smaller than a single group, the system degrades to returning only the system message rather than emitting an invalid request.

5.6 The agent loop & cost control

The loop sends the trimmed history to the model, reads any tool calls, executes them via dispatch, appends results, and repeats. After every model call it reads the exact token usage reported by the provider and accumulates the real cost using published per-token rates. Before each iteration it checks the accumulated cost against the cap; if exceeded, it aborts and reports the actual spend. Using the provider's reported usage — rather than a local estimate — makes cost accounting exact.

5.7 Caching layer

A SQLite database stores tool results keyed by a stable hash of the tool name and its *validated* arguments. Caching on validated arguments (after Pydantic fills defaults) normalizes cosmetically different but semantically identical calls into a single cache entry, raising the hit rate. Entries carry a creation timestamp and expire after a 24-hour time-to-live; expired entries are evicted on read. SQLite parameterized queries are used throughout, eliminating SQL-injection risk.

5.8 HTTP API

A FastAPI application wraps the agent with two endpoints: a `/health` liveness probe (used by container and hosting platforms) and a `/query` endpoint that accepts a question and returns the answer together with the list of tools the agent used. The API is stateless — each request is independent — which is the correct default for a horizontally scalable service; session-scoped memory is identified as future work.

6. Production Engineering

6.1 Graceful degradation

Tool failures are treated as data, not exceptions. A failed tool returns a structured error payload that the model is explicitly instructed to handle: it may rephrase and retry once, fall back to an alternate tool, or answer from general knowledge with a clear caveat. This behavior was verified repeatedly during development — for instance, when the Wikipedia backend timed out, the agent automatically fell back to web search and still produced a correctly cited answer.

6.2 Hard timeouts & the threading model

Each network tool's work is submitted to a module-level thread pool and awaited with a bounded timeout, guaranteeing a wall-clock ceiling on every tool call. The pool is deliberately module-level rather than wrapped in a context manager: a context-managed pool blocks on shutdown until its worker completes, which would defeat the timeout. Because threads cannot be forcibly killed in CPython, a timed-out worker is allowed to expire in the background while the application proceeds. A true hard kill of untrusted code would require a subprocess; this is documented as the boundary of the current approach.

6.3 Cost caps

Every request carries a hard monetary ceiling (default US$0.05). Because cost is accumulated from the provider's exact token usage, the cap is precise rather than estimated. This bounds the blast radius of a runaway loop or an adversarial prompt designed to force repeated expensive calls.

6.4 Caching strategy & staleness

The single global 24-hour TTL is the simplest correct policy. Its known limitation is staleness for volatile queries: a "latest news" result cached for 24 hours may return yesterday's news. The documented next step is differentiated TTLs — short for volatile sources (news), long for stable ones (encyclopedic articles, published papers) — and optionally keying on the recency parameter.

6.5 Security posture

Surface	Mitigation
Calculator code execution	numexpr evaluator only; `eval()` never used.
SQL injection (cache)	Parameterized queries exclusively.
Secret management	API key loaded from environment / `.env`; never committed; excluded from the Docker image.
Untrusted tool arguments	All model-supplied arguments validated against Pydantic schemas before use.
Unbounded cost	Per-request cost cap and turn cap.

7. Evaluation

7.1 Methodology & design decisions

The evaluation measures tool-selection accuracy: for each labeled question, did the agent invoke the expected tool? This metric is objective (no judge model required), cheap, and directly tied to the agent's core responsibility — routing. The harness runs each of 25 questions through the real agent, captures the tools actually called, and scores against the expected tool(s). Three deliberate design decisions govern scoring:

- **"Present among," not exclusivity.** A question is correct if the expected tool is among those called. A legitimate comparison question may call several tools; penalizing that would measure restraint rather than routing.
- **"None" means no research tool fired.** Questions answerable from the model's own knowledge are labeled `none` and are correct only if no tool was called.
- **Acceptable alternatives are allowed.** A question may list multiple acceptable tools (e.g. Wikipedia or web); any one suffices. To keep the metric falsifiable, `none` is forbidden from co-existing with real tools in a label — a rule the harness enforces by rejecting malformed labels.

7.2 Results

```
TOOL-SELECTION ACCURACY: 24/25 = 96.0%

calculator                  5/5 = 100%
search_web                  5/5 = 100%
search_arxiv                5/5 = 100%
search_wikipedia            4/4 = 100%
fetch_youtube_transcript    3/3 = 100%
none                        1/2 =  50%
```

7.3 What the eval surfaced

The harness earned its place by finding issues in the system and in its own specification, not merely confirming expectations:

- **A genuine tool bug.** The calculator raised an error when an expression referenced the constant `pi`, because the underlying evaluator does not define mathematical constants. This is a concrete defect the eval caught that ad-hoc testing had missed.
- **A specification ambiguity.** The question "What is the capital of France?" was labeled `none`, but the agent looked it up on Wikipedia. Both behaviors are defensible — parametric recall versus grounded, cited retrieval — which is why that row scores 50%. The value of the eval here is that it forced an explicit decision about whether grounding stable facts is desirable or wasteful, rather than leaving the behavior unexamined.

> **Principle.** An evaluation that only confirms existing beliefs is theater. One that surfaces defects in the implementation and disagreements between the specification and reality is doing the real work — it locates exactly where design, labels, and behavior diverge.

8. Deployment

8.1 Containerization

The application ships as a Docker image built on a slim Python base. Dependency manifests are copied and installed before the application code, exploiting layer caching so that code changes do not trigger dependency reinstalls. Secrets and local cache files are excluded from the image. The container binds to all interfaces so it is reachable when hosted, and exposes the FastAPI service via uvicorn.

8.2 Continuous integration

A GitHub Actions workflow runs on every push and pull request, installing dependencies on a clean machine and verifying that the package imports and compiles. This catches broken imports and syntax errors before they reach a deployment. Running the full evaluation in CI (which requires an API key as a managed secret) is identified as a future enhancement.

8.3 Hosting

The container is deployed to a managed platform that auto-detects the Dockerfile, injects the API key as a secret environment variable, and uses the `/health` endpoint as its liveness probe. On free-tier hosting the service may cold-start after inactivity, introducing a one-time delay on the first request after idle.

9. Limitations & Future Work

Area	Current state	Planned improvement
Streaming	Full answer returned at once	Token streaming to reduce time-to-first-token
HTTP memory	`/query` is stateless	Session-scoped memory keyed by a session identifier
Rate limiting	None (mitigated by caching)	Per-tool outbound rate limiting
Tool concurrency	Tool calls within a turn run sequentially	Parallel execution of independent calls (single-layer)
Cache policy	Single global 24h TTL	Differentiated TTLs per tool type
Observability	Debug-gated stderr tracing	LangSmith tracing and CI-gated regression eval

10. Engineering Decisions & Problems Encountered

This section records the substantive technical decision points encountered during development — architecture, system design, concurrency, security, and evaluation integrity. Each entry states the problem, the decision taken, and the reasoning. Trivial issues (typos, import names) are deliberately excluded; the focus is on decisions that materially shaped the system.

10.1 Tool-calling reliability & prompt design

The model answered from stale training data instead of invoking a tool

Problem. Asked for current regulatory news, the agent produced a confident answer prefixed "as of October 2023" — drawn entirely from training data — without ever calling the web-search tool. Tool selection is statistical, and with only a thin tool description and no system guidance, the model fell back on its priors.

Decision. Strengthen the tool description with explicit trigger phrases (news, current events, prices, "who currently holds a role"), and add a date-anchored system prompt that declares the training data stale and mandates a search for time-sensitive queries.

Rationale. The tool description and system prompt are the two levers that steer a statistical routing decision. Making the "when to search" rule explicit converts an unreliable judgment into a dependable one. This was the single most important behavioral fix in the project.

Routing reliably across five overlapping tools

Problem. With five tools whose remits partially overlap (web vs. Wikipedia vs. arXiv), the model risked ambiguous selection.

Decision. Author sharp, contrastive descriptions ("academic preprints, not general web"; "a specific given video, not discovery") and a system-prompt hierarchy ("most specific tool wins").

Rationale. Selection accuracy degrades as tool descriptions blur together. Contrastive wording gives the model the discriminating signal it pattern-matches against — later validated at 96% selection accuracy.

10.2 System design & data contracts

Heterogeneous tool outputs broke citation and consistency

Problem. Each tool initially returned a different shape (web hits, papers, articles, transcripts), forcing the model to guess where the source URL lived for each, and making programmatic citation extraction impossible.

Decision. Introduce a uniform `ToolResult` envelope (title, snippet, url, source, timestamp) returned by every retrieval tool, serialized as a JSON array.

Rationale. One output contract makes citations trivial, freshness explicit, and the underlying provider swappable without changing the model-facing interface. This is the same pattern used by production retrieval systems.

Hand-written schemas vs. a single source of truth

Problem. Tool argument schemas were first hand-authored as raw JSON, and in the wrong provider dialect (the Anthropic `input_schema` shape rather than the OpenAI `parameters` shape). Hand-maintaining schemas across five tools is error-prone and duplicative.

Decision. Define every tool's arguments as a Pydantic model and generate the provider schema programmatically; the same model also validates incoming arguments at runtime.

Rationale. One class becomes the single source of truth for both the schema the model sees and the validation the application enforces — eliminating drift and giving type safety and automatic argument validation for free.

Stubs that masqueraded as real data

Problem. During an incremental refactor, unimplemented tools returned placeholder strings resembling real output (e.g. "[search_arxiv stub] query=..."), which the model would synthesize on top of as though they were genuine results.

Decision. Make every stub return an explicit `[NOT IMPLEMENTED]` marker, treated by the system prompt as a failure signal.

Rationale. A stub that looks like data is worse than an error — it silently corrupts answers. An unambiguous failure marker keeps the model honest about what it does and does not have.

10.3 Concurrency & reliability

The hard-timeout illusion

Problem. A first attempt wrapped each tool call in a context-managed thread pool with a result timeout. The timeout fired correctly, but on exit the context manager called `shutdown(wait=True)`, which blocks until the worker finishes — so the function could still take far longer than the advertised ceiling. The timeout was effectively a lie.

Decision. Use a single module-level thread pool with no context manager; on timeout, return immediately and let the orphaned worker expire in the background.

Rationale. Threads cannot be forcibly killed in CPython, so a true wall-clock ceiling requires not waiting on the worker. A genuine hard kill of untrusted code would need a subprocess — documented as the boundary of the approach.

Nested thread pools risked starvation deadlock

Problem. An optimization to run a turn's tool calls concurrently introduced a second thread pool that drove the fan-out, while each tool already submitted its own work to the inner pool. Two stacked pools of bounded size can starve one another under load — outer tasks occupying workers that inner timeout tasks need.

Decision. Revert to single-layer sequential tool execution; record parallel tool execution (done at one layer) as a future enhancement.

Rationale. Concurrency must be reasoned about at one layer, not bolted onto an existing one. On a deadline, a correct sequential implementation beats a faster one carrying a deadlock risk.

Tool failure as data, not exception

Problem. Without explicit handling, a single flaky tool (a network blip, a rate-limit, a timeout) would raise and crash the entire request.

Decision. Every tool catches its own failures and returns a uniform error payload; the system prompt instructs the model to retry once, fall back to another tool, or answer with a caveat.

Rationale. Treating failures as structured data the model can read turns a crash into graceful degradation — verified when a Wikipedia timeout was automatically recovered by falling back to web search.

10.4 Memory & cost control

Unbounded history would break the context window and inflate cost

Problem. Naively appending every message forever eventually exceeds the context window and, because the full history is re-sent every turn, makes cost grow with conversation length.

Decision. A token-budgeted sliding window that always retains the system message, keeps newest messages first, and — critically — trims assistant tool-call messages together with their tool-result messages as indivisible groups.

Rationale. The group invariant is mandatory: the provider rejects a tool-result message lacking its preceding tool-call. Dropping whole groups (never partial ones) keeps every request valid while bounding tokens and cost.

Turns are not cost

Problem. A turn cap bounds the number of loop iterations but not the money spent — a single turn with a large context can cost more than several small turns.

Decision. Accumulate the exact per-request cost from the provider's reported token usage and enforce a hard monetary cap independent of the turn cap.

Rationale. Cost, not iteration count, is the quantity that must be bounded for predictable spend and protection against adversarial loop-forcing prompts.

10.5 Caching

Cache-key normalization and never caching failures

Problem. Keying the cache on the model's raw arguments would miss: the model might omit a defaulted field one call and include it the next, producing two entries for identical work. Separately, caching an error would replay a transient failure for the entire TTL.

Decision. Key on the *validated* arguments (after defaults are filled), and store results only when they are not errors.

Rationale. Validated-argument keys collapse equivalent calls into one entry, raising the hit rate; refusing to cache errors prevents a momentary outage from becoming a day-long one.

10.6 Security

A calculator built on `eval()` is a remote-code-execution hole

Problem. An initial calculator used Python's `eval()` with emptied builtins, under the mistaken belief it was sandboxed. This is a well-known escape vector — attribute traversal from a literal can reach arbitrary classes and execute code — and the argument regex permitted exactly the characters such an attack needs.

Decision. Replace `eval()` with the `numexpr` evaluator, which parses a restricted numeric grammar and never executes Python.

Rationale. Schema validation is the outer defense; a safe evaluator is the inner one. Both are required. Model-supplied arguments are untrusted by definition, so no execution path may reach arbitrary Python.

10.7 Evaluation integrity

An unfalsifiable metric measures nothing

Problem. Allowing a label to list both "no tool expected" and a real tool would make every outcome score as correct — the metric could never report failure.

Decision. Forbid "none" from co-existing with real tools in any label, and have the harness reject such labels at load time.

Rationale. A metric that cannot fail is theater. Enforcing falsifiability at the dataset level keeps the accuracy number meaningful.

An out-parameter silently severed from its caller

Problem. The evaluation initially scored 8% when the true figure was 96%. The cause: the agent function rebound its tool-capture list with `x = x or []`, which — because an empty list is falsy — replaced the caller's list with a fresh internal one, so every recorded tool name was discarded.

Decision. Remove the rebinding; rely on the explicit `is not None` guard already present.

Rationale. This is the same family of defect as the mutable-default trap — a falsy value passed by reference is silently replaced. The lesson is to distinguish "not provided" from "provided but empty," which only an explicit `None` check does. The diagnostic insight came from reading the trace: the tool-debug lines proved the tools fired, so the fault had to be in capture, not routing.

10.8 Engineering process

Refactoring from memory reintroduced solved bugs

Problem. When rewriting a working module from memory rather than from the working file, previously-fixed defects returned: a superseded import, the hard-timeout context-manager bug, and a dropped error-handling clause in the system prompt that caused the agent to retry-loop on tool failures.

Decision. Refactor from the last known-good source, layering new structure on top, rather than reconstructing from recollection — and re-run the prior tests against the refactored result to confirm behavior is preserved.

Rationale. A refactor must be behavior-preserving; the only way to know it is to verify against the same tests the original passed. Empirical verification — running the failure paths, not only the happy path — is a deliberate engineering practice, not an afterthought, because production never exercises those paths in development.

11. Conceptual Review: Design Q&A

At each stage of development, the design was probed with conceptual questions intended to test understanding of *why* the implementation works, not merely that it does. They are recorded here with answers and reasoning, as a reference to the thinking behind the system.

11.1 Tool calling & ReAct

> **Q. If a base model without native tool-calling support replaced the current model and the agent ran unchanged, what would break and why?**

A. The structured tool-call channel would come back empty: base models are neither trained to emit tool calls nor instruction-tuned, so the loop would return the model's first ungrounded answer and no tool would ever run. Recovering tool use would require a ReAct-style textual protocol plus a parser to extract actions from free text.

> **Q. To see the model's reasoning for choosing a tool, is ReAct required?**

A. No. With native tool calling, the rationale can be obtained by either instructing the model to include a one-line justification in its message content, or — more robustly — adding a required "reason" field to the tool schema so every call carries its justification by contract. A system-prompt instruction is cheapest but least enforced; a schema field is contract-guaranteed.

> **Q. In a ReAct loop, the model emits an action name followed by prose instead of a structured input. What happens?**

A. The input-parsing pattern misses, and most parsers raise a parsing exception and feed the error back so the model self-corrects; if it cannot recover within the retry budget, the turn fails. Native tool calling is immune to this class of fault because the SDK validates the structured arguments before the application sees them.

11.2 Conversation memory

> **Q. Why does the memory trim oldest messages first rather than newest?**

A. Recency correlates with relevance — the most recent turns are the active reasoning thread the model is currently working with. Dropping them would sever the train of thought. The system message is always preserved; newer groups are retained within the token budget; the oldest are evicted first.

> **Q. For a single turn in which the model calls two tools, what is the exact sequence of message roles, and which messages must stay grouped during trimming?**

A. The sequence is: system → user → assistant (with two tool calls) → tool → tool → assistant (final). The assistant tool-call message and both following tool-result messages must be trimmed as one indivisible group, because the provider rejects any tool-result message that does not immediately follow its matching tool call.

> **Q. What happens if the token budget is smaller than a single tool-call group?**

A. That entire group is dropped — partial groups are never emitted, since they would break tool-call pairing. In the worst case the returned history contains only the system message: degraded, but never an invalid request.

11.3 Cost control

> **Q. Why is the provider's reported usage more accurate than a local token-count estimate?**

A. The reported usage comes from the provider's own tokenizer and billing path, so it reflects exactly the tokens charged. A local estimate approximates only the message content and omits the serialized tool-call payloads and per-message formatting overhead, running optimistic by a noticeable margin.

> **Q. Should the cost cap be checked before or after the model call?**

A. Conceptually both: estimate before to avoid issuing an over-budget request, and verify after with the exact reported usage. The implementation checks at the top of each iteration using accumulated prior cost, which suffices here because a single inexpensive model call cannot overshoot the cap in one step — a deliberate choice that keeps the code and the mental model aligned.

> **Q. On aborting mid-loop, should a partial answer be returned or discarded?**

A. If a valid final answer has already been produced, return it. If the abort occurs during tool-calling with no final answer yet, discard the incomplete state and return a clear cost-cap message that includes the actual spend.

11.4 Caching

> **Q. Why key the cache on validated arguments rather than the raw arguments the model sent?**

A. Validated arguments have defaults filled and types normalized, so calls that differ only cosmetically (an omitted default versus an explicit one) collapse to a single cache key — materially raising the hit rate.

Q. Is caching the calculator worthwhile?

A. Only marginally. Local numeric evaluation completes in microseconds, so a disk round-trip can be slower than simply recomputing. The cache earns its value on network tools, where a hit avoids a multi-second call; for cheap local tools it is, at best, harmless.

Q. What is the risk of a 24-hour cache on "latest news" queries, and how would a production system address it?

A. Staleness — a result cached for a day may serve yesterday's news as today's. Production systems use differentiated time-to-live values (short for volatile sources, long for stable ones), key on the recency parameter, or bypass the cache entirely for recency-sensitive queries.

11.5 Evaluation design

Q. How should a question that legitimately calls multiple tools be scored?

A. By "present among," not exclusivity: the answer is correct if an expected tool is among those called. Requiring exclusivity would wrongly fail any legitimate comparison or multi-hop question. Over-calling is a separate, lesser concern that can be tracked independently.

Q. How is a "no tool expected" question scored?

A. It is correct only if no research tool fired. Infrastructure or bookkeeping tools are excluded from the check so that, for example, an internal memory write does not spuriously invalidate a "none" expectation.

Q. How are acceptable alternatives encoded?

A. The expected tool is normalized to a list, and the answer is correct if any listed tool was called. "None" is forbidden from appearing alongside real tools in a label, because that combination would make the metric unfalsifiable.

12. Appendix: Glossary & File Index

Glossary

Term	Definition
Tool calling	The mechanism by which an LLM emits a structured request to invoke an application-defined function.
Agentic loop	A loop in which the model repeatedly observes tool results and decides the next action until producing a final answer.
ToolResult envelope	The uniform output structure (title, snippet, url, source, timestamp) returned by all retrieval tools.
Token budget	A ceiling on the number of tokens of conversation history sent to the model per request.
Cost cap	A hard monetary ceiling per request, enforced from the provider's reported token usage.
TTL	Time-to-live; the duration a cached entry remains valid before expiry.
Graceful degradation	Continuing to provide useful service when a component (here, a tool) fails.
Tool-selection accuracy	The fraction of evaluation questions for which the agent invoked an expected tool.

File Index

Path	Purpose
`app/config.py`	Constants, timeouts, thread pool, OpenAI client, env loading.
`app/schemas.py`	Pydantic argument models, ToolResult envelope, error payload helper.
`app/memory.py`	Token counter and ConversationMemory sliding window.
`app/cache.py`	SQLite tool-result cache with 24h TTL.
`app/tools/web.py`	DuckDuckGo web search tool.
`app/tools/arxiv.py`	arXiv academic search tool.
`app/tools/wiki.py`	Wikipedia summary tool.
`app/tools/youtube.py`	YouTube transcript tool.
`app/tools/calc.py`	Sandboxed numexpr calculator.
`app/tools/registry.py`	Schema conversion, TOOLS list, dispatch_tool.
`app/agent/prompts.py`	System prompt construction.
`app/agent/executor.py`	The agentic loop and cost control.
`app/api/main.py`	FastAPI wrapper (/health, /query).
`scripts/agent_cli.py`	CLI and interactive REPL entry point.
`eval/dataset.jsonl`	25 labeled evaluation questions.
`eval/tool_accuracy.py`	Tool-selection accuracy harness.

End of report.